Summer of Science

Computer Vision

Endterm report 2025

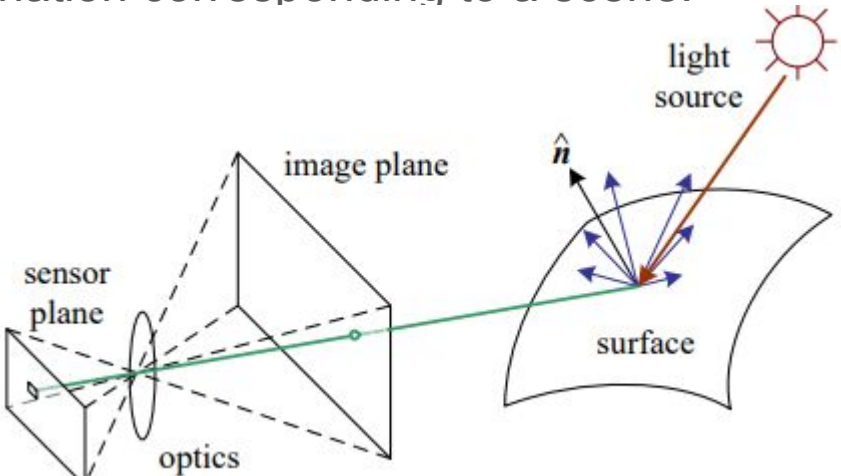
By prem kumar lodhi(24B1847)
Under Snigdha Reddy

Content

1. Photometric image formation
2. Feature Detection
3. Image Segmentation
4. Structure from Motion (SfM)
5. Computational photography
6. Object and face recognition

1.Photometric Image Formation

When light hits an object's surface, it is scattered and reflected. Many different models have been developed to describe this interaction. In this section, we first describe the most general form, the bidirectional reflectance distribution function, and then look at some more specialized models, including the diffuse, specular, and Phong shading models. We also discuss how these models can be used to compute the global illumination corresponding to a scene.



1.1The Bidirectional Reflectance Distribution Function (BRDF)

Imagine a flashlight shining on a surface and a camera observing the reflected light. The BRDF tells you **how bright the surface appears** to the camera **from that angle**, given the incoming light direction.the BRDF is a fourdimensional function that describes how much of each wavelength arriving at an incident direction vˆi is emitted in a reflected direction vˆr. The function can be written in terms of the angles of the incident and reflected directions relative to the surface frame as

$$f_r(\omega_i, \omega_o) = \frac{dL_r(\omega_o)}{dE_i(\omega_i)}$$

units of **inverse steradians**

- fr is the BRDF
- dLr is the reflected radiance in direction ωo
- dEi is the incoming irradiance from direction ωi

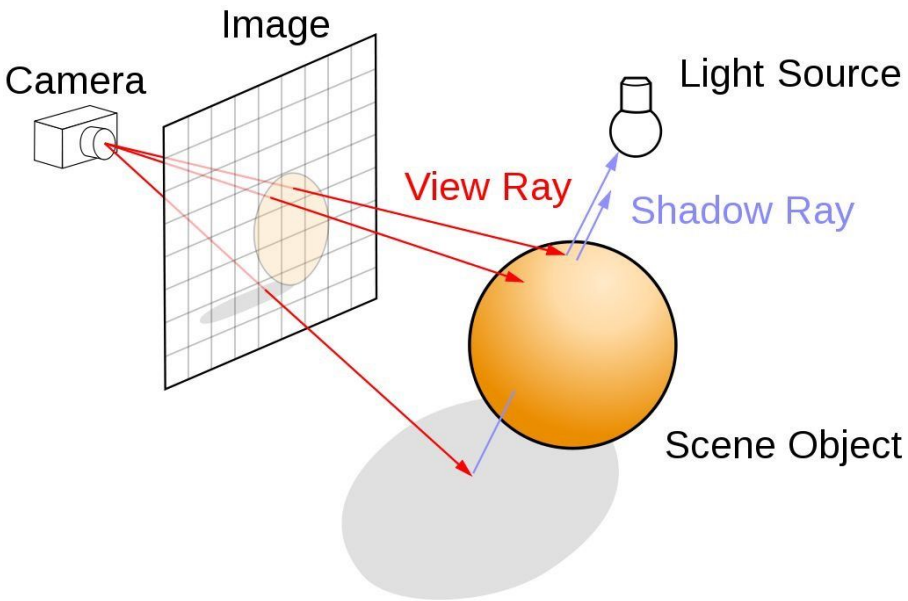
1.2.Matte Reflection

scatters light uniformly in all directions and is the phenomenon we most normally associate with shading, e.g., the smooth (non-shiny) variation of intensity with surface normal that is seen when observing a statue Diffuse reflection also often imparts a strong body color to the light since it is caused by selective absorption and re-emission of light inside the object's material. the amount of light depends on the angle between the incident light direction and the surface normal θi . This is because the surface area exposed to a given amount of light becomes larger at oblique angles, becoming completely self-shadowed as the outgoing surface normal points away from the light

1.3.Ray traycing

Ray tracing is a rendering technique that can realistically simulate the lighting of a scene and its objects by rendering physically accurate reflections, refractions, shadows, and indirect lighting. Ray tracing generates computer graphics images by tracing the path of light from the view camera (which determines your view into the scene), through the 2D viewing plane (pixel plane), out into the 3D scene, and back to the light sources. As it traverses the scene, the light may reflect from one object to another (causing reflections), be blocked by objects (causing shadows), or pass through transparent or semi-transparent objects (causing refractions). All of these interactions are combined to produce the final color and illumination of a pixel that is then displayed on the screen. This reverse tracing process of eye/camera to light source is chosen because it is far more efficient than tracing all light rays emitted from light sources in multiple directions.

Another way to think of ray tracing is to look around you, right now. The objects you're seeing are illuminated by beams of light. Now turn that around and follow the path of those beams backwards from your eye to the objects that light interacts with. That's ray tracing.The primary application of ray tracing is in computer graphics, both non-real-time (film and television) and real-time (video games). Other applications include those in architecture, engineering, and lighting design.



2.Feature Detection

Feature detection and matching are an essential component of many computer vision applications.The first kind of feature that you may notice are specific locations in the images, such as mountain peaks, building corners, doorways, or interestingly shaped patches of snow. These kinds of localized feature are often called keypoint features or interest points (or even corners) and are often described by the appearance of patches of pixels surrounding the point location.

Another class of important features are edges, e.g., the profile of mountains against the sky. These kinds of features can be matched based on their orientation and local appearance (edge profiles) and can also be good indicators of object boundaries and occlusion events in image sequences. Edges can be grouped into longer curves and straight line segments, which can be directly matched or analyzed to find vanishing points and hence internal and external camera parameters.

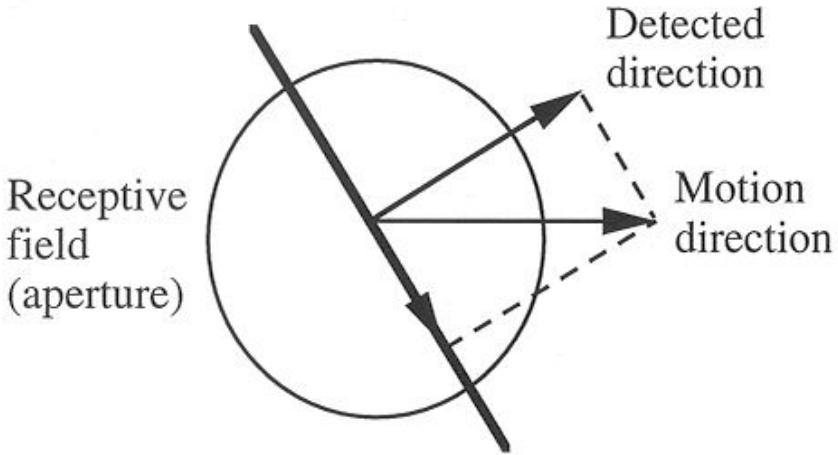
2.1.Point Feature Detection

There are two main approaches to finding feature points and their correspondences. The first is to find features in one image that can be accurately tracked using a local search technique, such as correlation or least squares. The second is to independently detect features in all the images under consideration and then match features based on their local appearance. The former approach is more suitable when images are taken from nearby viewpoints or in rapid succession (e.g., video sequences), while the latter is more suitable when a large amount of motion or appearance change is expected, e.g., in stitching together panoramas, establishing correspondences in wide baseline stereo, or performing object recognition.

we split the keypoint detection and matching pipeline into three separate stages. During the feature detection (extraction) stage (Section 2.1.1), each image is searched for locations that are likely to match well in other images. At the feature description stage (Section 2.1.2), each region around detected keypoint locations is converted into a more compact and stable (invariant) descriptor that can be matched against other descriptors.Feature matching stage (Section 2.1.3) efficiently searches for likely matching candidates in other images.

2.1.1 The Feature Detection (extraction) stage

textureless patches are nearly impossible to localize. Patches with large contrast changes (gradients) are easier to localize, although straight line segments at a single orientation suffer from the aperture problem, as shown schematically in Figure, i.e., it is only possible to align the patches along the direction normal to the edge direction.Gradients in at least two (significantly) different orientations are the easiest to localize.



Steps to detect feature points location

1. Compute the horizontal and vertical derivatives of the image Ix and Iy by convolving the original image with derivatives of Gaussians.

2. Compute the three images corresponding to the outer products of these gradients. (The matrix A is symmetric, so only three entries are needed.)

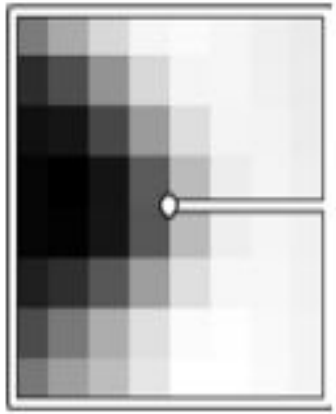
3. Convolve each of these images with a larger Gaussian.

4. Compute a scalar interest measure using one of the formulas discussed above.

5. Find local maxima above a certain threshold and report them as detected feature point locations.

2.1.2.The Feature Description Stage

After detecting features (keypoints), we must match them, i.e., we must determine which features come from corresponding locations in different images. In some situations, e.g., for video sequences (Shi and Tomasi 1994) or for stereo pairs that have been rectified, the local motion around each feature point may be mostly translational. In this case, simple error metrics, such as the sum of squared differences or normalized cross-correlation, can be used to directly compare the intensities in small patches around each feature point.Because feature points may not be exactly located, In most cases, however, the local appearance of features will change in orientation and scale, and sometimes even undergo affine deformations. Extracting a local scale, orientation, or affine frame estimate and then using this to resample the patch before forming the feature descriptor is thus usually preferable (as shown in Figure). Even after compensating for these changes, the local appearance of image patches will usually still vary from image to image.



MOPS descriptors are formed using an 8 × 8 sampling of bias and gain normalized intensity values, with a sample spacing of five pixels relative to the detection scale. This low frequency sampling gives the features some robustness to interest point location error and is achieved by sampling at a higher pyramid level than the detection scale.

2.1.3.Feature matching stage

Once we have extracted features and their descriptors from two or more images, the next step is to establish some preliminary feature matches between these images.The first is to select a matching strategy, which determines which correspondences are passed on to the next stage for further processing. The second is to devise efficient data structures and algorithms to perform this matching as quickly as possible.Once we have decided on a matching strategy, we still need to search efficiently for potential candidates. The simplest way to find all corresponding feature points is to compare all features against all other features in each pair of potentially matching images. Unfortunately, this is quadratic in the number of extracted features, which makes it impractical for most applications. A better approach is to devise an indexing structure, such as a multi-dimensional search tree or a hash table, to rapidly search for features near a given feature.

Such indexing structures can either be built for each image independently (which is useful if we want to only consider certain potential matches, e.g., searching for a particular object) or globally for all the images in a given database, which can potentially be faster, since it removes the need to iterate over each image. For extremely large databases (millions of images or more), even more efficient structures based on ideas from document retrieval can be used.

Recognizing objects in a cluttered scene.Two of the training images in the database are shown on the left. These are matched to the cluttered scene in the middle using SIFT features, shown as small squares in the right image. The affine warp of each recognized database image onto the scene is shown as a larger parallelogram in the right image.

2.2 Edge Detection

While interest points are useful for finding image locations that can be accurately matched in 2D, edge points are far more plentiful and often carry important semantic associations. For example, the boundaries of objects, which also correspond to occlusion events in 3D, are usually delineated by visible contours. Other kinds of edges correspond to shadow boundaries or crease edges, where surface orientation changes rapidly. Isolated edge points can also be grouped into longer curves or contours, as well as straight line segments.

Think of an image as a height field. On such a surface, edges occur at locations of steep slopes, or equivalently, in regions of closely packed contour lines (on a topographic map). A mathematical way to define the slope and direction of a surface is through its gradient,

$$\boldsymbol{J}(\boldsymbol{x}) = \nabla I(\boldsymbol{x}) = (\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y})(\boldsymbol{x}).$$

the local gradient vector J points in the direction of steepest ascent in the intensity function. Its magnitude is an indication of the slope or strength of the variation, while its orientation points in a direction perpendicular to the local contour. Unfortunately, taking image derivatives accentuates high frequencies and hence amplifies noise, since the proportion of noise to signal is larger at high frequencies. It is therefore prudent to smooth the image with a low-pass filter prior to computing the gradient. Because we would like the response of our edge detector to be independent of orientation, a circularly symmetric smoothing filter is desirable.the Gaussian is the only separable circularly symmetric filter and so it is used in most edge detection algorithms.

Coloured Edge Detection

While most edge detection techniques have been developed for grayscale images, color images can provide additional information. For example, noticeable edges between iso-luminant colors (colors that have the same luminance) are useful cues but fail to be detected by grayscale edge operators. One simple approach is to combine the outputs of grayscale detectors run on each color band separately.7 However, some care must be taken. For example, if we simply sum up the gradients in each of the color bands, the signed gradients may actually cancel each other! (Consider, for example a pure red-to-green edge.) We could also detect edges independently in each band and then take the union of these, but this might lead to thickened or doubled edges that are hard to link. A better approach is to compute the oriented energy in each band, e.g., using a second-order steerable filter, and then sum up the orientation-weighted energies and find their joint best orientation. Unfortunately, the directional derivative of this energy may not have a closed form solution (as in the case of signed first-order steerable filters), so a simple zero crossing-based strategy cannot be used. However, the technique described by Elder and Zucker (1998) can be used to compute these zero crossings numerically instead.

Combined brightness, color, texture boundary detector. Successive rows show the outputs of the brightness gradient (BG), color gradient (CG), texture gradient (TG), and combined (BG+CG+TG) detectors. The final row shows human-labeled boundaries derived from a database of hand-segmented images

An alternative approach is to estimate local color statistics in regions around each pixel. This has the advantage that more sophisticated techniques (e.g., 3D color histograms) can be used to compare regional statistics and that additional measures, such as texture, can also be considered. Above Figure shows the output of such detectors.

Resources used till now

https://developer.nvidia.com/discover/ray-tracing

https://www.youtube.com/@Eigensteve

https://www.youtube.com/@DigitalSreeni

Book of CV by Richard Szeliski

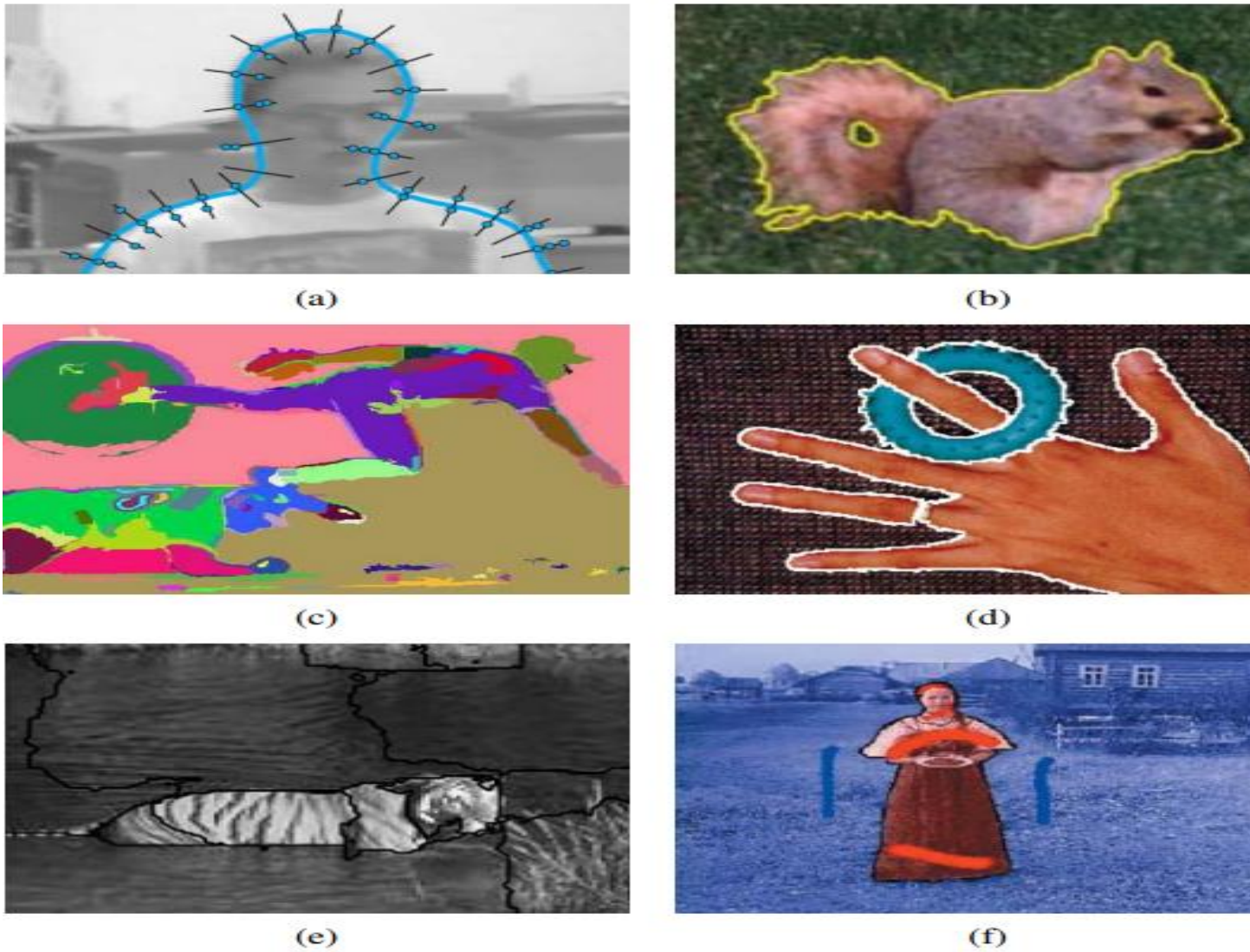
seas.ucla.edu

google.com

3.Image Segmentation

In the realm of computer vision, one of the most critical low- and mid-level tasks is **image segmentation** — the process of partitioning an image into distinct regions that are homogeneous according to some criterion, such as intensity, color, or texture. Segmentation provides a foundation for higher-level tasks like object recognition, tracking, and scene understanding. Among the diverse segmentation techniques, **split and merge**, **mean shift and mode finding**, and **normalized cuts** represent three conceptually and algorithmically distinct families of approaches.So in the following text we'll explore some segmentation

techniques.



Som popular image segmentation techniques: (a) active contours; (b) level sets; (c) graph-based merging; (d) mean shift; (e) texture and intervening contour-based normalized cuts; (f) binary MRF solved using graph cuts.

Split and Merge

Split and merge is a **region-based segmentation** technique, developed to overcome limitations of simpler methods such as region growing and thresholding. Where thresholding globally classifies pixels based on intensity and region growing locally expands regions from seeds, split and merge adopts a more systematic, hierarchical strategy.

The idea is to treat the entire image as a single region and then recursively **split** it into smaller regions if it fails to satisfy a homogeneity criterion. Conversely, adjacent regions that are similar are **merged** to avoid unnecessary over-segmentation. The result is a segmentation that respects the local uniformity of image properties while maintaining spatial coherence.

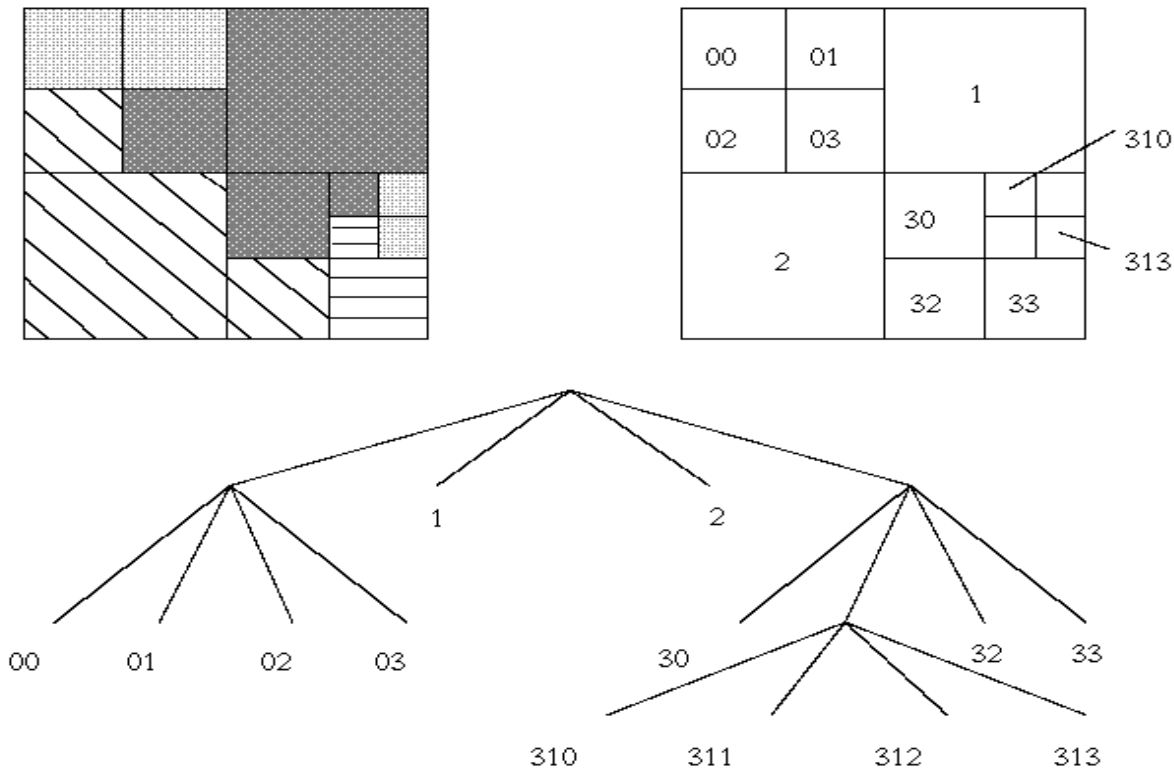


Figure 5.45 *Segmentation quadtree.*

pyramid levels or do not have the same parent) that can be merged into a homogeneous region, merge them.

4. Merge small regions with the most similar adjacent region if it is necessary to remove small-size regions.

Theoretical Formulation

Consider an image defined over a domain DDD. We seek to partition DDD into NNN regions RiR_iRi such that: each Region is connected. Reagion1∩Region2 = ∅

each Region satisfies a homogeneity criterion, such as having variance of intensity below a threshold τ.

Algorithm outline

Step 1: Initialization : Begin with the entire image as a single region.

Step 2: Splitting : If the region does not satisfy the homogeneity criterion, split it into four equal quadrants (or another scheme if desired).Recursively test and split each resulting subregion until all regions satisfy the criterion.

Step 3: Merging :Examine neighboring regions that share a boundary.Merge them if the combined region still satisfies the homogeneity criterion.

The splitting process naturally lends itself to a quadtree representation, where each node represents a region and has up to four child nodes corresponding to the quadrants. The merging step ensures that the final segmentation is not over-partitioned.

Advantages

Systematic and hierarchical: ensures coverage of the entire domain without dependence on initial seeds. Enforces both region homogeneity and connectivity. Simple to implement and interpret.

Limitations

Can produce blocky, unnatural boundaries. It is also Sensitive to noise, small variations within regions may trigger unnecessary splits.

Mean Shift and Mode Finding

Mean shift is a non-parametric, iterative clustering technique that identifies high-density regions (modes) in feature space. Originally developed for tracking in video sequences, it has since found wide application in image segmentation. Unlike parametric methods such as k-means that assume a fixed number of clusters and specific cluster shapes, mean shift adapts to the intrinsic structure of the data.

In segmentation, each pixel is represented as a point in a multi-dimensional feature space (e.g., intensity, color, texture, spatial coordinates). The algorithm moves each point toward the nearest mode of the estimated probability density function (pdf), effectively grouping pixels that converge to the same mode into the same region.

Mean shift is based on kernel density estimation (KDE). Given n data points {ui}, the density at a point u is estimated as:

$$f_K(\mathbf{u}) = \frac{1}{nh^d} \sum_{i=1}^{n} K(\frac{\mathbf{u} - \mathbf{u}_i}{h})$$

where:

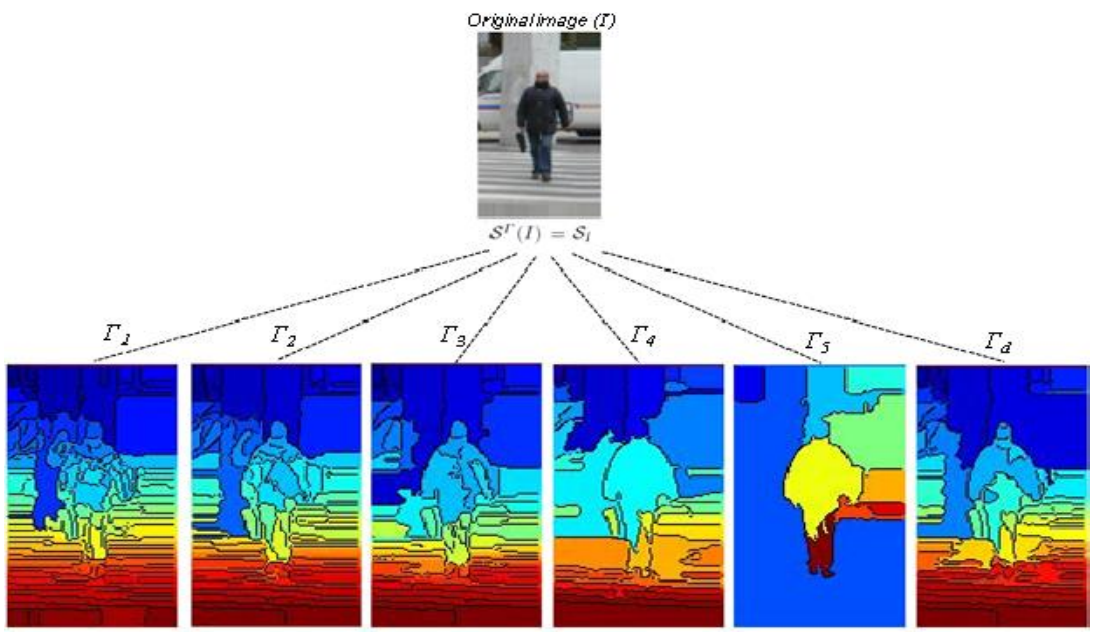
K is a kernel function (often Gaussian or Epanechnikov). h is the bandwidth (scale) parameter. d is the dimensionality of the feature space.

The mean shift vector at x(u in prev formula) is:

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_{i=1}^{n} \mathbf{x}_i \cdot K'\left(\frac{\|\mathbf{x}-\mathbf{x}_i\|}{h}\right)}{\sum_{i=1}^{n} K'\left(\frac{\|\mathbf{x}-\mathbf{x}_i\|}{h}\right)} - \mathbf{x}$$

where K' is the derivative of K.

By iteratively moving each point x in the direction of m(x), the point converges to a mode of the density.



Segmentations with varying bandwidths—showing how region granularity changes with scale.

Algorithm Outline

Step 1 Initialization : For each data point, initialize its position in feature space.

Step 2 Iterative Update : Compute the mean shift vector. Move the point by this vector. Repeat until convergence to a mode.

Step 3 Clustering : after convergence, assign points to the same cluster if they converge to the same mode.

Advantages

Non-parametric: no need to specify the number of clusters. Can detect arbitrarily-shaped clusters. Naturally combines spatial and appearance features, yielding coherent segmentation.

Limitations

Computationally expensive for large datasets. Sensitive to choice of bandwidth h: too small produces many spurious clusters, too large merges distinct regions. May not perform well in highly textured or noisy images.

Normalized Cuts

Normalized cuts is a graph-theoretic segmentation method that formulates image segmentation as a graph partitioning problem. Pixels (or regions) are represented as nodes in a graph, and edges encode similarity between them. The goal is to partition the graph into disjoint subgraphs such that the similarity across the cut is low and the similarity within each partition is high.A naive minimum cut algorithm tends to produce trivial, small cuts. Normalized cuts overcome this by introducing a normalization term that favors balanced, meaningful partitions.

Consider a graph G=(V,E), with vertex set V and weighted edges wij. Partition V into two disjoint sets A and B. Define the cut cost:

$$\mathrm{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}$$

This cost alone favors separating out small, weakly connected subsets. To address this, Shi and Malik proposed the **normalized cut (Ncut)** criterion:

$$\mathrm{Ncut}(A,B) = \frac{\mathrm{cut}(A,B)}{\mathrm{assoc}(A,V)} + \frac{\mathrm{cut}(A,B)}{\mathrm{assoc}(B,V)}$$

where assoc(A,V) is the total connection of A to all nodes:

$$\mathrm{assoc}(A,V) = \sum_{i \in A, j \in V} w_{ij}$$

Minimizing Ncut balances between minimizing dissimilarity across the cut and maintaining internal similarity within partitions.

Algorithm Outline

Step 1 Graph : Construction Represent the image as a graph, with pixels as nodes. Define edge weights wij based on similarity of intensity, color, texture, and spatial proximity.

Step 2: Eigenvalue Problem Form the degree matrix D (diagonal, with Dii=∑j wij) and the affinity matrix W.Solve the generalized eigenvalue problem:

$$(D - W)\mathbf{y} = \lambda D\mathbf{y}$$

The eigenvector corresponding to the second smallest eigenvalue provides an approximate solution to the Ncut minimization.

Step 3: Recursive Partitioning Use the solution to partition the graph into two sets.Repeat recursively to obtain finer segmentations.

Advantages

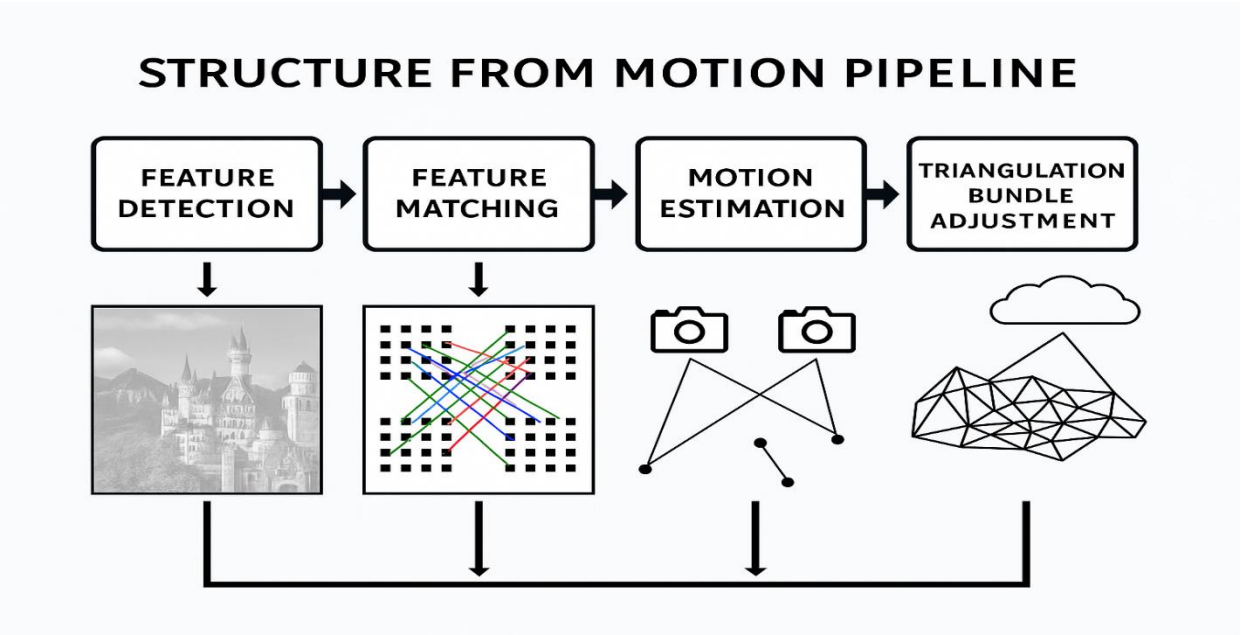
Produces globally consistent, balanced segmentations. Incorporates both local and global image information.Robust to noise and can handle textured regions.

Limitations

Computationally intensive for large images (requires eigen-decomposition). Requires constructing and storing large matrices. Sensitive to parameter settings for weight computation.

4.Structure from Motion (SfM)

the process of recovering the 3D structure of a scene and the motion of the camera by analyzing multiple 2D images taken from different viewpoints. Here we examine the fundamental principles behind SfM and related methods, such as triangulation, two-frame motion estimation, constrained motion models, and factorization techniques. These methods are key to applications such as 3D modeling, augmented reality, autonomous navigation, and robotics.



SfM addresses the question: *Given a set of 2D point correspondences across multiple images, how can we recover both the 3D positions of those points and the camera motion parameters?* This is possible because the 2D image projections constrain the possible 3D structure and camera motion, but the problem is inherently ambiguous if too few views are used.

SfM pipelines generally include four stages:

1. Detect and match feature points (e.g., SIFT, SURF).
2. Estimate camera motion between frames.
3. Recover 3D point positions using triangulation.
4. Refine both camera and point estimates using **bundle adjustment**, which minimizes reprojection error.

Now we will discuss few methods through which we can execute structure from motion.

Triangulation

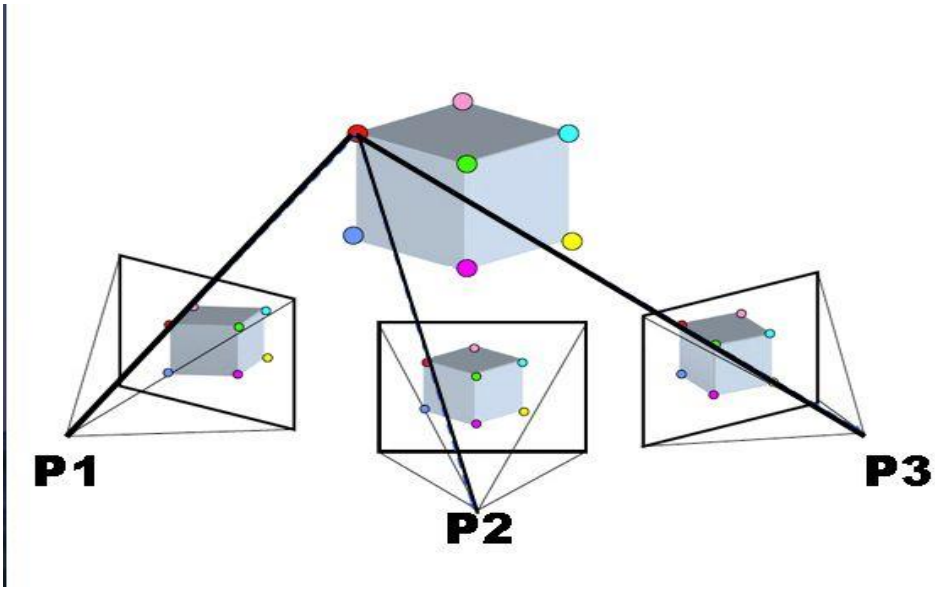
It is the process of estimating the 3D position of a point from its projections in two or more images. Once the relative camera poses are known, each 2D observation defines a ray in 3D space, and the intersection of these rays (or the point minimizing the distance to them) gives the estimated 3D point.

Mathematically, if a 3D point X projects to x_1 and x_2 in two views, and the camera projection matrices are P1 and P2, we solve for X such that:

$$x_1=P_1X, \qquad x_2=P_2X$$

Since rays may not intersect perfectly (due to noise), least-squares methods or midpoint approximations are used.

Triangulation is the fundamental building block for recovering 3D structure point-by-point once the camera motion is known. Eg of it in the given image below.



Two-frame Structure from Motion

When only two images are available, SfM becomes more challenging because fewer constraints are present. Still, it is possible to recover the 3D structure up to a scale factor.

The key idea is to compute the essential matrix E (for calibrated cameras) or the fundamental matrix F (for uncalibrated cameras) from point correspondences.E encodes the relative rotation R and translation t of the camera between the two views:

$$E=[t]_{\times}R$$

Once E or F is estimated (e.g., via the eight-point algorithm or RANSAC), the relative motion can be decomposed, and triangulation can be applied to recover the 3D points. However, there is an inherent ambiguity in the scale of translation, and often only the direction is recovered in monocular setups.

This two-frame SfM serves as the foundation for more general multi-view reconstructions and is often used in stereo vision.

Constrained Structure and Motion

In practice, additional constraints on the camera motion or scene structure can be exploited to improve accuracy and reduce ambiguity. These constraints may come from prior knowledge or mechanical setups. Common examples include:

1. **Planar scenes:** If the scene is known to be planar, homographies can describe the mapping between views.
2. **Known motion models:** For example, if the camera moves only along a known path (e.g., rotating around a fixed axis), the number of free parameters in motion estimation is reduced.
3. **Rigid motion:** Assuming that all points belong to a rigid object reduces the number of unknowns significantly.
4. **Known camera intrinsics:** If calibration parameters are known, the fundamental matrix estimation can be replaced with essential matrix computation, simplifying the estimation.

Such constraints help make the estimation problem well-posed and more robust to noise and outliers.

Factorization

Factorization methods offer an elegant linear algebraic solution to the SfM problem under orthographic or weak-perspective projection assumptions. They are particularly effective in multi-frame settings with many points tracked across many views.

The key idea is that the 2D measurement matrix W of tracked feature coordinates can be factorized into a motion matrix M and a shape matrix S using Singular Value Decomposition (SVD):

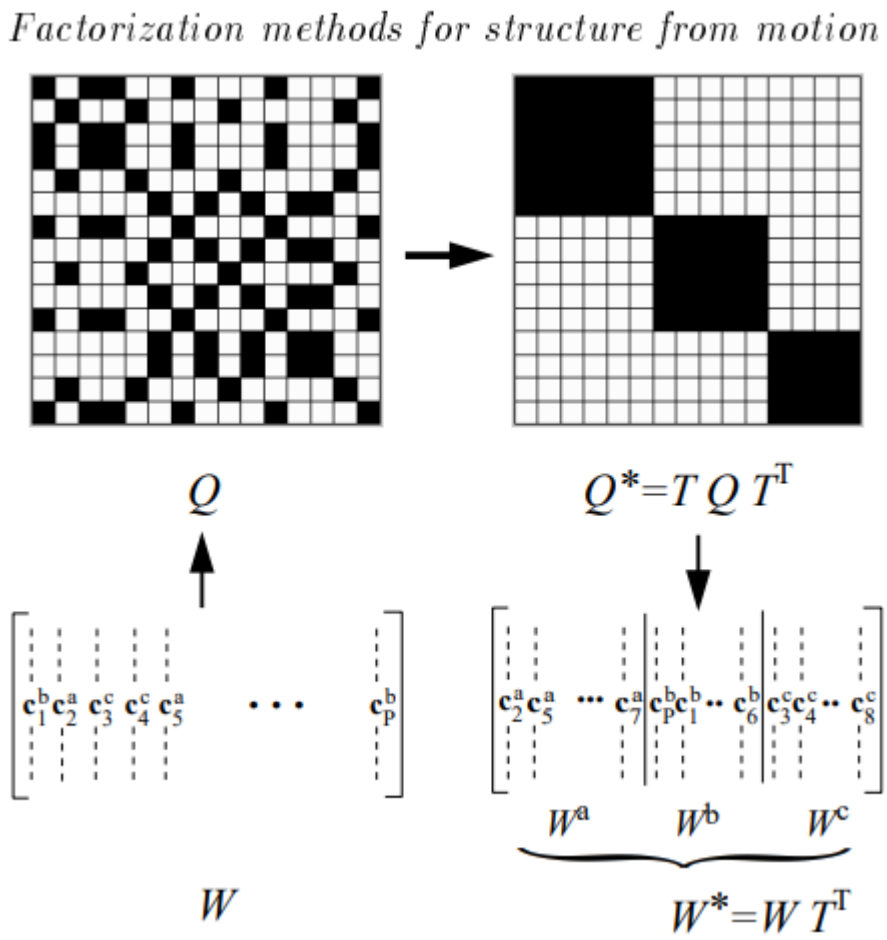
$$W=MS$$

Here:

- W is $2m\times n$, where m is the number of frames and n is the number of tracked points.
- M contains the camera parameters (rotation, translation).
- S contains the 3D coordinates of the scene points.

Since the rank of W is at most 3 (under orthographic projection), SVD allows us to approximate it and recover M and S up to an affine ambiguity, which can later be refined.

Factorization is attractive because it is a batch method, works well for many frames and points, and is less sensitive to noise compared to incremental methods. However, it assumes good feature tracking and minimal occlusions.



5.Computational photography

Computational photography is a field that combines principles of how light interacts with objects, how sensors record incoming light, and how algorithms can reconstruct or enhance what is captured. Traditional cameras are inherently limited — they cannot simultaneously capture the full range of brightness in a scene, they blur fine details because of optical imperfections or motion, and their sensors respond in ways that distort true brightness. These imperfections lead to images that fail to represent the scene as it

truly appeared. Computational photography addresses these shortcomings by explicitly modeling how a scene is transformed into pixel values and then solving the reverse problem to recover an accurate or perceptually better representation of the scene. In effect, the recorded image is interpreted as a combination of measurable factors, which can then be adjusted to correct distortions, recover lost information, and improve detail and clarity. This makes the final image closer to what human vision would perceive, and sometimes even beyond what our eyes can resolve.

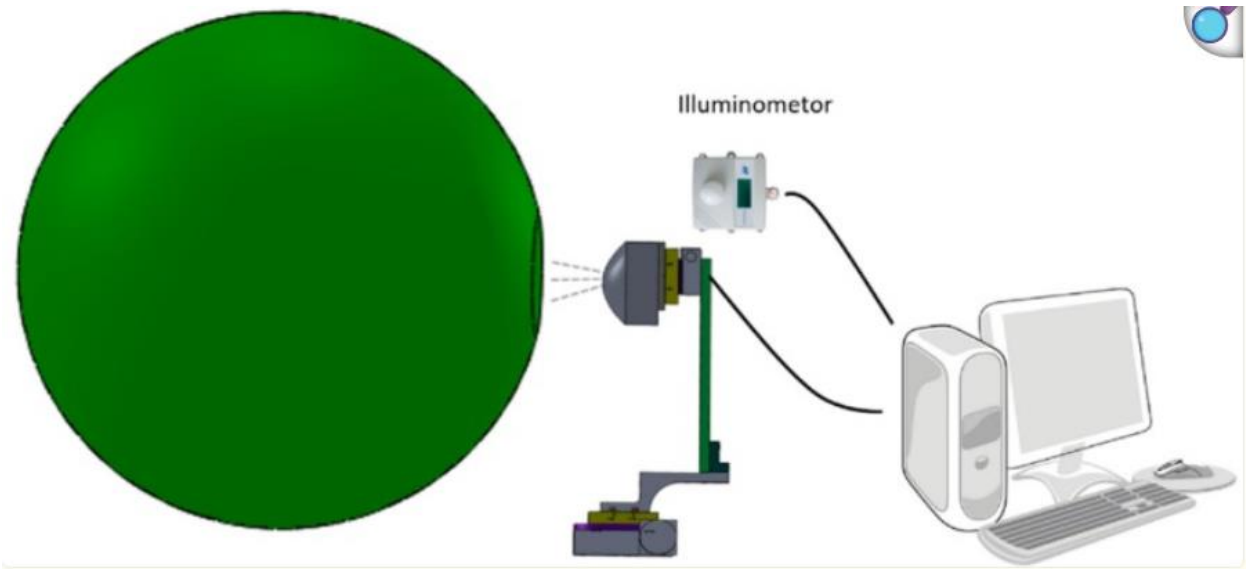
Photometric Calibration

One of the first challenges is that camera sensors are nonlinear(lens are usually curved): doubling the amount of light in the scene does not result in doubling the recorded intensity. This creates inconsistencies, making it difficult to merge exposures or determine the true scene brightness. Photometric calibration addresses this by determining the camera's response curve and reconstructing the real irradiance map of the scene. The process involves photographing a static scene at multiple exposure times. The recorded intensity $I_k(x)$ at pixel x for exposure time Δt_k relates to true irradiance E(x) via:

$$I_k(x) = f(E(x) \times \Delta t_k)$$

where f() is the unknown camera response function. By taking logarithms, the relation becomes approximately linear:

$$\ln I_k(x) = g(E(x)) + \ln \Delta t_k$$



Open in a new tab

Photometric calibration with an integrating sphere.

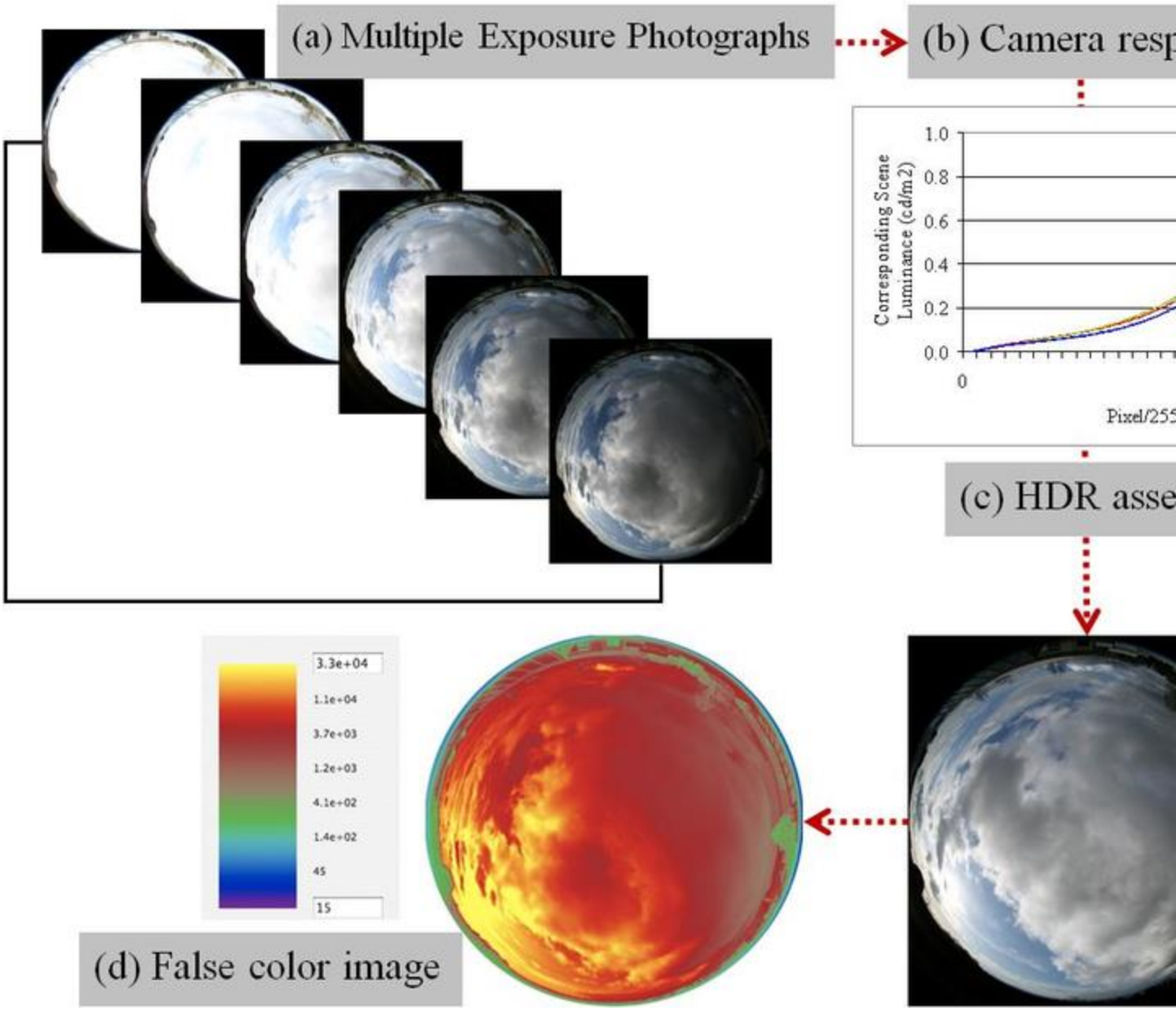
where g = ln f. This produces a system of equations for each pixel observed at different exposures. Solving this system while enforcing smoothness of g yields both the response curve and the irradiance map E(x). The irradiance map is a physically meaningful representation of the scene's brightness distribution, unaffected by sensor distortions. This step is fundamental because it enables subsequent techniques — like HDR imaging — to operate on accurate data.

High Dynamic Range Imaging

Even with calibration, a single exposure cannot capture very bright highlights and deep shadows at the same time. High Dynamic Range (HDR) imaging solves this by combining multiple calibrated exposures, each optimized for different brightness levels of the scene. For each pixel, the irradiance estimate is calculated as a weighted average of its values in different exposures:

$$\ln E(x) = [\Sigma_k\, w(I_k(x))\, (g(I_k(x)) - \ln \Delta t_k)] \,/\, \Sigma_k\, w(I_k(x))$$

The weight function w(I) assigns higher weights to well-exposed pixels and lower weights to pixels that are saturated or underexposed. The resulting irradiance map spans a much wider range of brightness than a single image could.



Since display devices cannot show such a wide range, a tone mapping operator T is used:

$$I_d(x) = T(E(x))$$

This compresses brightness intelligently while preserving fine detail in both highlights and shadows. The HDR process enables images to convey much more of the richness and contrast present in the real-world scene, avoiding washed-out skies and dark, featureless shadows.

Super-Resolution and Blur Removal

Cameras are limited in how much fine detail they can resolve because of sensor resolution, lens quality, and motion. Super-resolution aims to recover a sharper, higher-resolution image by combining several slightly offset low-resolution images L_k. The model assumes each L_k is a blurred, downsampled, and noisy version of an unknown high-resolution image H:

$$L_k = D_k B_k H + n_k$$

where D_k is the downsampling operator, B_k models blur, and n_k is noise. To find H, one minimizes a cost function that balances data fidelity and smoothness:

$$\text{minimize_}H \left(\Sigma_k ||L_k - D_k B_k H||^2 + \lambda||\nabla H||^1\right)$$

This produces a high-resolution image with sharper edges and finer textures, recovering details that would otherwise be lost.

For blur removal, the blurry image I is modeled as a convolution of the sharp scene S with a blur kernel K:

$$I = K * S + n$$

Once K is estimated, S can be recovered by deconvolution in the frequency domain:

$$\hat{S}(u,v) = \hat{I}(u,v) / \hat{K}(u,v)$$

Regularization is used to suppress noise and artifacts. These techniques restore edge sharpness and reveal textures obscured by motion blur, defocus, or camera shake, improving the clarity and detail of the image.

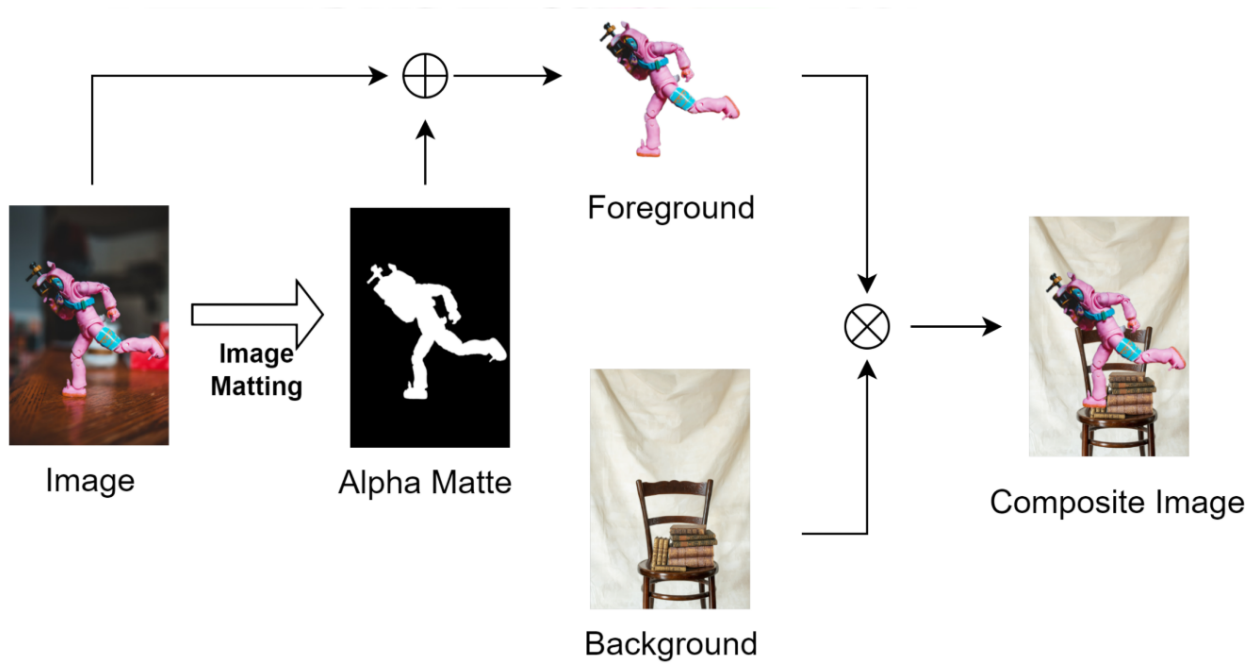
Image Matting and Compositing

Separating a subject from its background — especially when the edges are soft, semi-transparent, or mixed — is a complex problem. Each observed pixel I is assumed to be a weighted mixture of foreground F, background B, and transparency α:

$$I = \alpha F + (1-\alpha)B$$

Here α varies between 0 and 1 and indicates the degree to which a pixel belongs to the foreground. Since this equation contains more unknowns than measurements, additional assumptions are made, such as smooth variation of α in regions where colors blend gradually. The solution minimizes an energy function that balances fitting the observed

colors and enforcing smoothness of α:



$$E(\alpha) = \Sigma_x ||I_x - (\alpha_x F_x + (1-\alpha_x)B_x)||^2 + \beta \Sigma_{neighbors} (\alpha_x - \alpha_y)^2$$

The resulting α matte provides a per-pixel map of transparency, which allows for realistic compositing. The foreground F can then be blended over any new background B' using the same model:

$$I' = \alpha F + (1-\alpha)B'$$

This allows for seamless integration of subjects into different scenes, preserving fine details like hair strands and transparent regions.

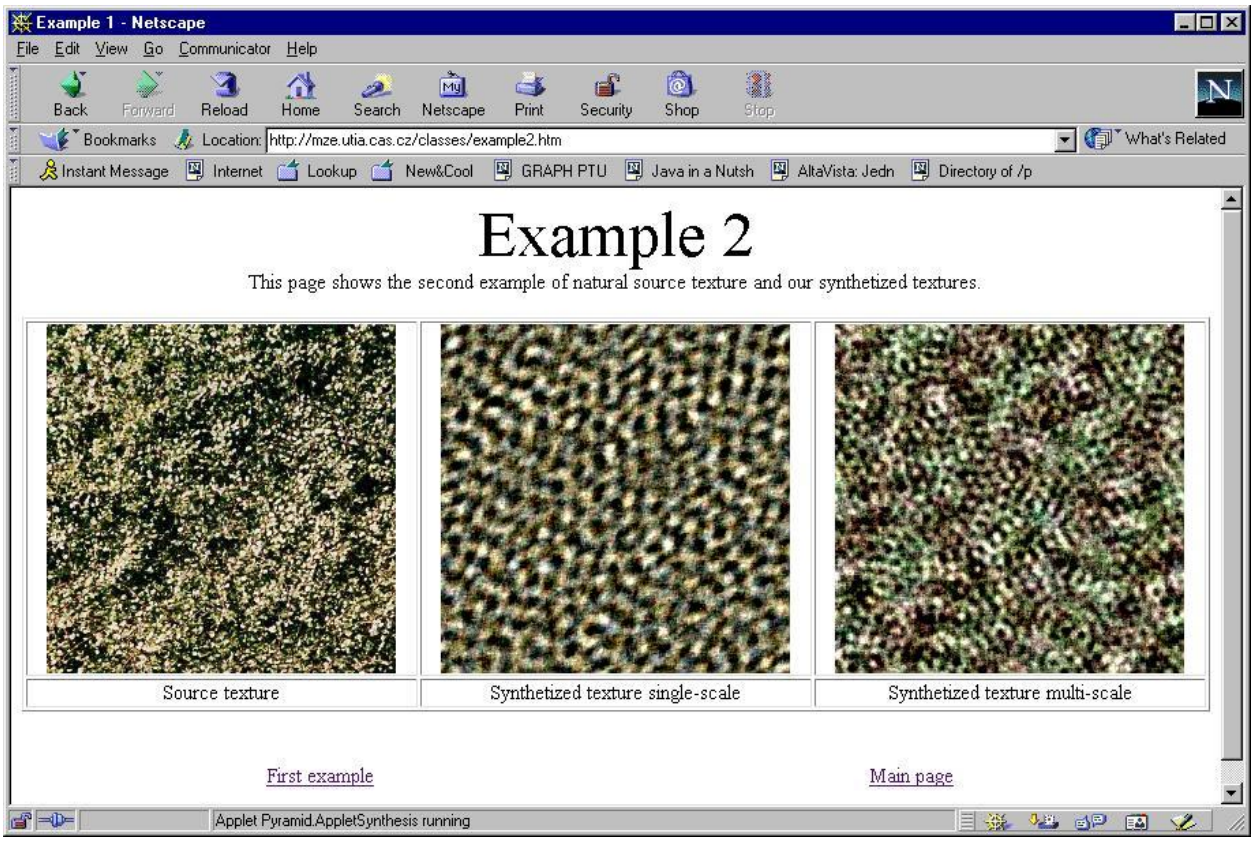
Texture Analysis and Synthesis

Many surfaces exhibit textures — repeating but slightly varying patterns — that are important for realism in images. Texture analysis involves measuring statistical properties of a texture, such as spatial correlations, frequency components, or histograms of local patterns. These descriptors help in identifying, comparing, and segmenting textures in an image.

Texture synthesis generates a new texture T' that is visually similar to a sample texture T. One method grows T' pixel by pixel by matching local neighborhoods to those in T. A more principled approach assumes textures can be described by probabilistic models like Markov Random Fields:

$$P(T) \propto \exp(-\Sigma_{cliques} \psi(C))$$

where ψ(C) describes how small groups of pixels interact locally. Sampling from this distribution produces T' with realistic structure. Multi-scale approaches synthesize textures at different resolutions to ensure both global coherence and local variation. These techniques are widely used in graphics, inpainting missing regions, and creating seamless backgrounds or surfaces.



Across all of these techniques, the goal is not just to model the imaging process but to invert it intelligently — to extract more information, correct imperfections, and produce images that are visually and physically closer to reality or even improved beyond what the camera alone can deliver. The formulations allow each computational step to contribute directly to clarity, realism, and visual appeal.

6.Object and face recognition

Object Recognition

Object and face recognition are among the most fundamental tasks in computer vision. They aim to assign a semantic label — “cat,” “car,” or even a person’s name — to a region of an image. While humans accomplish this effortlessly, for computers the challenge lies in the fact that the observed image I is only an indirect, noisy projection of the underlying scene, distorted by illumination, occlusion, and viewpoint changes. To overcome this, recognition systems explicitly model the probability of a class C given the observed image, written mathematically as:

$$\hat{C} = \text{argmax}_C \, P(C \mid I)$$

where $P(C \mid I)$ is the posterior probability of class C given image I. This follows directly from Bayes’ theorem, where one might also model the likelihood $P(I \mid C)$ and prior $P(C)$.

Practically, one rarely works directly with pixels: instead, the image I is transformed into a feature vector x, and then the probability is computed in feature space. In a deep learning framework, a neural network $f_\theta(I)$ outputs a vector of class probabilities p_i, and the system is trained to minimize the cross-entropy loss:

$$L = -\sum_i y_i \log p_i$$

where y_i is an indicator for the true class and p_i is the predicted probability.

Face Recognition

Face recognition, unlike generic object recognition, is more subtle: it involves distinguishing between many similar-looking individuals. Rather than training a network to predict one of thousands of identities directly, it is more effective to map each face image I into an embedding vector $z(I) \in \mathbb{R}^d$, designed so that embeddings of the same person are close together while embeddings of different people are far apart. One popular way to train such embeddings is the triplet loss, defined as:

$$L_{triplet} = \max(0, \|z_a - z_p\|^2 - \|z_a - z_n\|^2 + \alpha)$$

Here z_a is the embedding of an “anchor” image, z_p is an image of the same person (positive), and z_n is an image of a different person (negative). The idea is simple: force the distance between z_a and z_p to be smaller than the distance between z_a and z_n by at least a

margin α. The choice of distance metric $||\cdot||_2$ reflects Euclidean geometry in embedding space, but cosine similarity is also commonly used.

Face Detection

Face detection is the task of finding regions of an image where faces are likely to appear. In mathematical terms, we want to estimate the location parameters of bounding boxes B = [x, y, w, h] for each face and also compute a confidence score s ∈ [0,1] representing the probability that a given box contains a face. Formally, the detection problem is posed as maximizing:

$$\hat{B}, \hat{s} = \text{argmax}_{B,s} P(B, s \mid I)$$

Classical detectors, like Viola–Jones, used Haar-like rectangular features ϕ(I,B) computed efficiently using integral images. These features were fed into a cascade of weak classifiers that were trained using AdaBoost. In effect, they approximated the posterior P(face | B) using weighted sums of binary features.

Modern convolutional neural network (CNN) based detectors jointly learn features and predictors. During training, the loss function combines a classification loss L_cls and a regression loss L_reg to adjust both the confidence scores and the bounding box locations:

$$L = \lambda_{cls} L_{cls} + \lambda_{reg} L_{reg}$$

Here L_cls is typically a cross-entropy loss over “face” vs. “non-face” classes, and L_reg is a smooth L1 loss that penalizes deviations between the predicted and true bounding box coordinates.

Pedestrian Detection

Pedestrian detection is conceptually similar to face detection: the goal is to find all bounding boxes Bk in I where people appear. The primary difference lies in the shape and variability of the target class: humans are deformable, articulated objects, often occluded by other people or objects. Earlier approaches modeled the pedestrian silhouette using the Histogram of Oriented Gradients (HOG) feature:

$$x_{HOG} = \text{hist}(\nabla I)$$

Here ∇I is the image gradient field, and the histogram bins capture local edge orientations within each cell. These features are then fed into a linear classifier (like a support vector machine) to decide if a given window contains a pedestrian. The scoring function in such a system is typically:

$$S(B) = w^t x_{HOG}(B) + b$$

where w and b are the learned SVM parameters. Deep learning based pedestrian detectors replace hand-crafted HOG features with CNN-learned features $\phi(I,B)$, but the overall formulation remains the same: find Bk such that $s_k = P(\text{pedestrian} \mid B_k) > \tau$, with τ a confidence threshold.

Face Recognition with Identification and Verification

After detecting and localizing the face, recognition involves comparing the observed face to a gallery of known faces and either verifying or identifying the person. Each detected face I is converted into an embedding vector z(I), and similarity is computed using a distance function $d(\cdot,\cdot)$. For verification, we compare two embeddings:

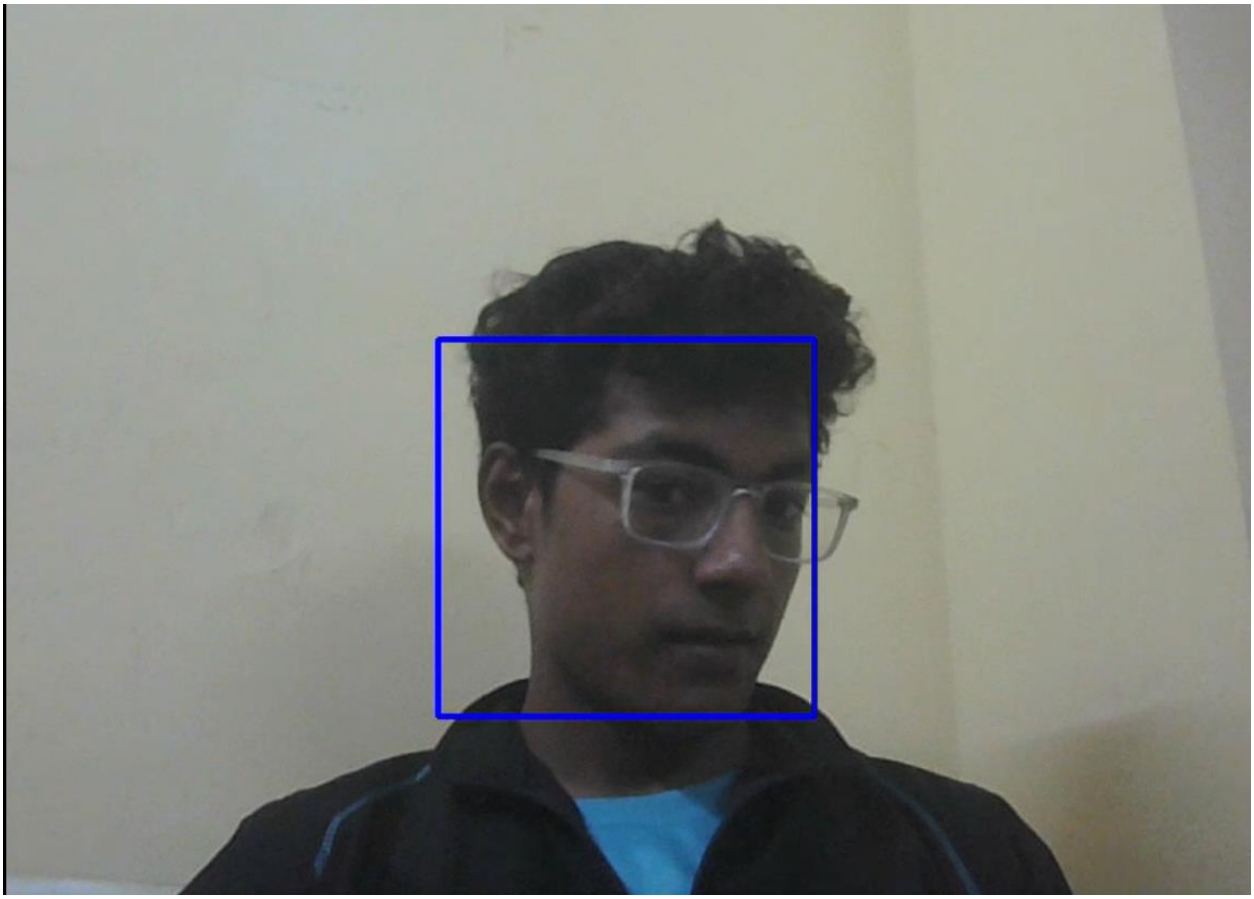
$$d(I_1, I_2) = ||z(I_1) - z(I_2)||_2$$

If $d(I_1, I_2) < \tau$, we accept that they belong to the same person; otherwise, they are deemed different. For identification, we find the gallery index j that minimizes the distance:

$$\hat{j} = \text{argmin}_j \, ||z(I) - z(G_j)||$$

where G_j are the stored gallery embeddings.

Advanced losses such as ArcFace improve the discrimination power of embeddings by enforcing a larger angular margin between different classes.



using opencv and haarcascade to make a live face detection module (me in the frame)

full implementation video and my code link:

https://drive.google.com/drive/folders/1QnBXocVEzAT4TMKwix2b_aamCrblFzqd?usp=sharing